

```
let name: string = "Jude";
let age: number = 22;
let isConnected: boolean = true;
let nothing: null = null;
let notDefined: undefined = undefined;
```

Tableau :

```
let users: string[] = ["Alice", "Bob"];
let numbers: number[] = [1, 2, 3];
```

Objet :

```
let user: {
  id: number;
  name: string;
} = {
  id: 1,
  name: "Jude"
};
```

Créer ses propres types

```
type Task = {
  id: number;
  content: string;
  done: number;
};
```

On peut créer un type :

```
type CreateUserBody = {
  name: string;
  mail: string;
  password: string;
};
```

```
const { name, mail, password } = request.body as CreateUserBody;
```

ypeScript côté backend avec NodeJS

3.1 Initialiser TypeScript

```
npm init -y  
npm install typescript --save-dev  
npx tsc --init
```

Compiler TypeScript
npx tsc -w

Exécuter le serveur

Après compilation :

```
node dist/index.js
```

Fastify
npm install fastify

Exemple minimal :

```
import Fastify from "fastify";  
  
const fastify = Fastify();  
  
fastify.get("/", (request, reply) => {  
  reply.send("Hello World !");  
});  
  
fastify.listen({ port: 8080 });
```

Exemple route GET

```
fastify.get("/users", (request, reply) => {  
  const users = [  
    { id: 1, name: "Alice", picture: "alice.png" },  
    { id: 2, name: "Bob", picture: "bob.png" }  
  ];  
  
  reply.send(users);  
});
```

Exemple route POST

```
type CreateUserBody = {
  name: string;
  mail: string;
  password: string;
};

fastify.post("/users", (request, reply) => {
  const { name, mail, password } = request.body as CreateUserBody;

  if (!name || !mail || !password) {
    reply.code(400).send({ error: "Champs manquants" });
    return;
  }

  reply.code(201).send();
});
```

Requêtes SQL avec SQLite

```
npm install better-sqlite3
npm install --save-dev @types/better-sqlite3
```

```
import Sqlite3 from "better-sqlite3";

const
```

Lire des données : SELECT

```
const query = db.prepare("SELECT * FROM users;");
const users = query.all();
```

```
reply.send(users);
```

```
const query = db.prepare("SELECT * FROM users WHERE mail = ?;");
const user = query.get(mail);
```

```
const query = db.prepare(`
  INSERT INTO users (name, mail, password)
  VALUES (?, ?, ?);
`);
```

```
const result = query.run(name, mail, password);
```

- utilisateur connecté : 1
- utilisateur avec qui il parle : 2
- SELECT *
- FROM messages
- WHERE
- (from_user = 1 AND to_user = 2)
- OR
- (from_user = 2 AND to_user = 1);
- const query = db.prepare(`
- SELECT
- messages.id,
- messages.from_user,
- messages.to_user,
- messages.message,
-
- sender.name AS sender_name,
- receiver.name AS receiver_name
-
- FROM messages
-
- INNER JOIN users AS sender
- ON messages.from_user = sender.id
-
- INNER JOIN users AS receiver
- ON messages.to_user = receiver.id
-
- WHERE
- (messages.from_user = ? AND messages.to_user = ?)
- OR
- (messages.from_user = ? AND messages.to_user = ?);
- `);
-
- const messages = query.all(
- connectedUserId,
- otherUserId,
- otherUserId,
- connectedUserId
-);

Modifier des données : UPDATE

```
const query = db.prepare(`
  UPDATE tasks
  SET content = ?, done = ?
  WHERE id = ?;
`);
```

```
query.run(content, done, id);
```

Supprimer des données : DELETE

```
const query = db.prepare("DELETE FROM tasks WHERE id = ?;");
query.run(id);
```

```
import Fastify from "fastify";
import cors from "@fastify/cors";
import Sqlite3 from "better-sqlite3";
```

```
const fastify = Fastify();
const db = Sqlite3("./database/db.sqlite");
```

```
fastify.register(cors);
```

```
type CreateTaskBody = {
  content: string;
};
```

```
type UpdateTaskBody = {
  content: string;
  done: number;
};
```

```
fastify.get("/tasks", (request, reply) => {
  try {
    const query = db.prepare("SELECT * FROM tasks;");
    const tasks = query.all();

    reply.code(200).send(tasks);
  } catch (error) {
    reply.code(500).send();
  }
});
```

```
fastify.post("/tasks", (request, reply) => {
  try {
    const { content } = request.body as CreateTaskBody;

    const query = db.prepare("INSERT INTO tasks (content, done) VALUES (?, ?);");
    const result = query.run(content, 0);

    reply.code(201).send({
      id: result.lastInsertRowid
    });
  } catch (error) {
    reply.code(500).send();
  }
});
```

```

    }
  });

  fastify.put("/tasks/:taskId", (request, reply) => {
    try {
      const { taskId } = request.params as { taskId: string };
      const { content, done } = request.body as UpdateTaskBody;

      const query = db.prepare(`
        UPDATE tasks
        SET content = ?, done = ?
        WHERE id = ?;
      `);

      query.run(content, done, Number(taskId));

      reply.code(204).send();
    } catch (error) {
      reply.code(500).send();
    }
  });

  fastify.delete("/tasks/:taskId", (request, reply) => {
    try {
      const { taskId } = request.params as { taskId: string };

      const query = db.prepare("DELETE FROM tasks WHERE id = ?;");
      query.run(Number(taskId));

      reply.code(204).send();
    } catch (error) {
      reply.code(500).send();
    }
  });

  fastify.listen({ port: 8080 });

  npm install @fastify/cors
  import cors from "@fastify/cors";

  fastify.register(cors);

```

Frontend Vanilla TypeScript avec Vite

9.1 Créer le projet frontend

```
npm create vite@latest front
cd front
npm install
npm run dev
```

Manipuler le DOM en TypeScript

```
<form id="task-form">
  <input id="task-input" type="text" />
  <button>Ajouter</button>
</form>
```

```
<ul id="task-list"></ul>
```

```
<form id="task-form">
  <input id="task-input" type="text" />
  <button>Ajouter</button>
</form>
```

```
<ul id="task-list"></ul>
```

API Fetch côté frontend

11.1 Fetch, c'est quoi ?

```
type Task = {
  id: number;
  content: string;
  done: number;
};
```

```
async function loadTasks(): Promise<Task[]> {
  const response = await fetch("http://localhost:8080/tasks");

  if (!response.ok) {
    throw new Error("Erreur lors du chargement des tâches");
  }

  const tasks = await response.json() as Task[];
  return tasks;
}
```

POST

```
async function createTask(content: string): Promise<void> {
  const response = await fetch("http://localhost:8080/tasks", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      content: content
    })
  });

  if (!response.ok) {
    throw new Error("Erreur lors de la création de la tâche");
  }
}
```

PUT : modifier une tâche

```
async function updateTask(task: Task): Promise<void> {
  const response = await fetch(`http://localhost:8080/tasks/${task.id}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      content: task.content,
      done: task.done
    })
  });

  if (!response.ok) {
    throw new Error("Erreur lors de la modification");
  }
}
```

DELETE

```
async function deleteTask(id: number): Promise<void> {
  const response = await fetch(`http://localhost:8080/tasks/${id}`, {
    method: "DELETE"
  });

  if (!response.ok) {
    throw new Error("Erreur lors de la suppression");
  }
}
```



```

type Task = {
  id: number;
  content: string;
  done: number;
};

const app = document.querySelector<HTMLDivElement>("#app");

if (!app) {
  throw new Error("Élément #app introuvable");
}

app.innerHTML = `
  <h1>TodoList</h1>

  <form id="task-form">
    <input id="task-input" type="text" placeholder="Nouvelle tâche" />
    <button type="submit">Ajouter</button>
  </form>

  <ul id="task-list"></ul>
`;

const form = document.querySelector("#task-form") as HTMLFormElement;
const input = document.querySelector("#task-input") as HTMLInputElement;
const list = document.querySelector("#task-list") as HTMLUListElement;

async function loadTasks(): Promise<Task[]> {
  const response = await fetch("http://localhost:8080/tasks");

  if (!response.ok) {
    throw new Error("Impossible de charger les tâches");
  }

  return await response.json() as Task[];
}

async function createTask(content: string): Promise<void> {
  const response = await fetch("http://localhost:8080/tasks", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ content })
  });

  if (!response.ok) {
    throw new Error("Impossible de créer la tâche");
  }
}

```

```
}
```

```
async function updateTask(task: Task): Promise<void> {  
  const response = await fetch(`http://localhost:8080/tasks/${task.id}`, {  
    method: "PUT",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({  
      content: task.content,  
      done: task.done  
    })  
  });  
};
```

```
if (!response.ok) {  
  throw new Error("Impossible de modifier la tâche");  
}  
}
```

```
async function deleteTask(id: number): Promise<void> {  
  const response = await fetch(`http://localhost:8080/tasks/${id}`, {  
    method: "DELETE"  
  });  
};
```

```
if (!response.ok) {  
  throw new Error("Impossible de supprimer la tâche");  
}  
}
```

```
async function renderTasks(): Promise<void> {  
  const tasks = await loadTasks();
```

```
  list.innerHTML = "";
```

```
  for (const task of tasks) {  
    const li = document.createElement("li");
```

```
    const checkbox = document.createElement("input");  
    checkbox.type = "checkbox";  
    checkbox.checked = task.done === 1;
```

```
    const span = document.createElement("span");  
    span.textContent = task.content;
```

```
    const deleteButton = document.createElement("button");  
    deleteButton.textContent = "Supprimer";
```

```
    checkbox.addEventListener("change", async () => {  
      await updateTask({  
        ...task,
```

```

        done: checkbox.checked ? 1 : 0
    });

    await renderTasks();
});

deleteButton.addEventListener("click", async () => {
    await deleteTask(task.id);
    await renderTasks();
});

li.appendChild(checkbox);
li.appendChild(span);
li.appendChild(deleteButton);

list.appendChild(li);
}
}

form.addEventListener("submit", async (event) => {
    event.preventDefault();

    const content = input.value.trim();

    if (content === "") {
        return;
    }

    await createTask(content);

    input.value = "";
    await renderTasks();
});

renderTasks();

```

Authentification avec JWT

npm install jsonwebtoken

npm install --save-dev @types/jsonwebtoken

import JWT from "jsonwebtoken";

```
const privateKey = "ma_cle_secrete";
```

```
const token = JWT.sign(
    { id: user.id, mail: user.mail },
    privateKey
);
const payload = JWT.verify(token, privateKey);
```

authorization Bearer Token

```
await fetch("http://localhost:8080/messages", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
    "Authorization": `Bearer ${token}`  
  },  
  body
```

```
// backend
```

```
const authorization = request.headers.authorization;
```

```
if (!authorization) {  
  reply.code(401).send();  
  return;  
}
```

```
const token = authorization.replace("Bearer ", "");
```